

TAREA 2

Desarrollo de un Sistema de Comunicación Inalámbrica con Arduino

1. Introducción y Definición del Problema

1.1. Contexto general

La comunicación inalámbrica constituye uno de los pilares fundamentales del desarrollo tecnológico moderno. Desde redes de sensores industriales hasta sistemas de IoT (*Internet of Things*), la capacidad de transmitir datos sin vínculo físico entre nodos ha permitido la proliferación de aplicaciones que serían inviables con infraestructura cableada. En particular, los módulos de radiofrecuencia en la banda de 433 MHz se emplean ampliamente en automatización del hogar, telemedición y control remoto de bajo costo, gracias a su bajo consumo energético, alcance de decenas de metros y disponibilidad en el mercado.

1.2. Problema de la transmisión en canales inalámbricos

A diferencia de un canal cableado, el canal inalámbrico es un **medio compartido y no confiable**. Los principales fenómenos que degradan la calidad de la señal son:

- **Ruido térmico (AWGN):** fluctuaciones aleatorias de voltaje presentes en todo receptor electrónico, modeladas habitualmente como ruido blanco gaussiano aditivo. Su efecto se cuantifica mediante la relación señal-ruido (*Signal-to-Noise Ratio*, SNR):

$$\text{SNR} = \frac{P_{\text{señal}}}{P_{\text{ruido}}} \quad [\text{adimensional}] \quad \text{SNR}_{\text{dB}} = 10 \log_{10} \left(\frac{P_s}{P_n} \right) \quad (1)$$

Un SNR bajo implica mayor probabilidad de error por bit (*Bit Error Rate*, BER).

- **Desvanecimiento (*fading*):** variación temporal de la potencia recibida causada por reflexiones, difracciones y absorciones de la señal al propagarse en el espacio. En entornos interiores, el desvanecimiento multitrayecto puede provocar cancelaciones destructivas que reducen momentáneamente la potencia recibida en varios dB.
- **Interferencia de banda:** los 433 MHz forman parte de la banda ISM (*Industrial, Scientific and Medical*), libre de licencia, por lo que coexisten en ella múltiples dispositivos como mandos a distancia, sensores meteorológicos y equipos de domótica, incrementando la probabilidad de colisiones y errores de bit.
- **Atenuación en espacio libre:** la potencia de una señal se atenúa proporcionalmente al cuadrado de la distancia, según la ecuación de Friis:

$$P_r = P_t G_t G_r \left(\frac{\lambda}{4\pi d} \right)^2 \quad (2)$$

donde P_t es la potencia transmitida, G_t y G_r las ganancias de antena, λ la longitud de onda y d la distancia entre nodos.

La consecuencia directa de estos fenómenos es la aparición de **errores de bit**: uno o más bits del paquete transmitido llegan al receptor con valor invertido. En ausencia de mecanismos de detección, el receptor no puede distinguir un paquete corrupto de uno válido, lo que hace infiable cualquier información recibida.

1.3. Justificación del protocolo propuesto

Ante la problemática anterior, el diseño de un protocolo de comunicación debe incorporar, como mínimo, los siguientes elementos:

1. **Sincronización y delimitación de trama:** un byte de cabecera de valor fijo (0xAA) permite al receptor identificar el inicio de cada paquete y sincronizar su ventana de lectura. Este patrón alternado (10101010 en binario) es especialmente favorable para la modulación ASK (*Amplitude Shift Keying*) que emplean los módulos de 433 MHz, ya que facilita la recuperación de reloj en el receptor.

2. **Direccionamiento:** los campos de ID emisor e ID receptor permiten operar múltiples nodos en el mismo canal, implementando un filtrado básico a nivel de protocolo. El receptor descarta paquetes cuyo ID destino no coincida con el propio, evitando el procesamiento de tramas ajenas.
3. **Carga útil estructurada:** la imagen binarizada de 32×32 píxeles se segmenta en bloques de 3 bytes (24 bits) por paquete. Esta fragmentación reduce la probabilidad de que un único error de canal corrompa más de una línea de la imagen, y adapta el tamaño del paquete a las limitaciones de los módulos de 433 MHz, cuyo rendimiento disminuye con tramas largas.
4. **Detección de errores mediante checksum/CRC:** el último byte del paquete contiene un valor de verificación calculado a partir de los seis bytes anteriores. En este caso, el checksum se obtiene mediante la suma módulo 256 de dichos bytes:

$$CRC = \left(\sum_{i=1}^6 B_i \right) \text{ mód } 256$$

donde B_i representa cada uno de los bytes de datos del paquete. En el receptor se vuelve a calcular este valor y se compara con el CRC recibido. Si ambos valores difieren, se asume que uno o más bytes fueron alterados durante la transmisión, permitiendo **detectar** (aunque no corregir) el error y descartar el paquete corrupto. La capacidad de detección de este mecanismo es cercana al 99.6 % para errores aleatorios de un byte.

En conjunto, estos mecanismos conforman un protocolo liviano y adecuado para microcontroladores con recursos limitados como Arduino, donde la memoria RAM disponible es del orden de 2 KB y la velocidad de procesamiento no supera los 16 MHz. La implementación práctica de este protocolo sobre hardware real constituye el objetivo central del presente laboratorio.

2. Objetivos

- Familiarizarse con la comunicación inalámbrica usando módulos de 433 MHz (TX y RX).
- Construir un protocolo simple para enviar y recibir una imagen binarizada representada en una matriz.

3. Materiales

- 2 placas Arduino (uno como emisor y otro como receptor).
- Módulos de radio 433 MHz (TX y RX).
- Cables de conexión y protoboard.
- LEDs o monitor serial para validación.

4. Descripción general del protocolo básico

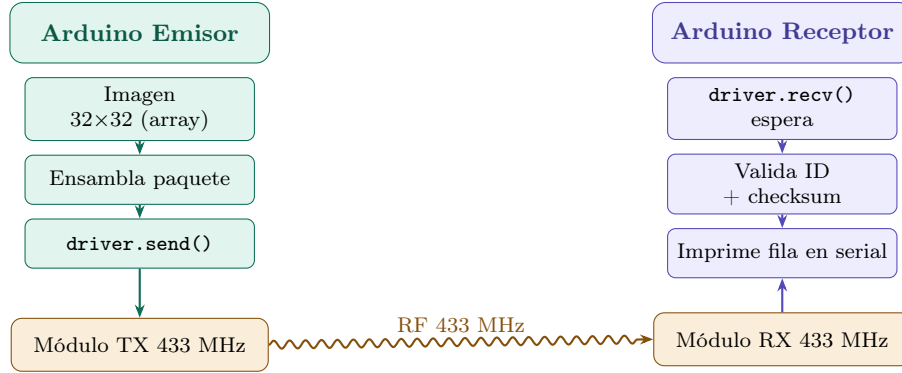
La Tabla 1 describe la estructura del paquete de comunicación empleado en el laboratorio. El paquete tiene un tamaño total de 7 bytes.

5. Arquitectura del sistema propuesta

La Figura 1 ilustra la arquitectura completa del sistema, mostrando los dos nodos Arduino, los módulos RF de 433 MHz, el flujo de datos interno de cada nodo y la estructura del paquete transmitido.

Cuadro 1: Estructura del paquete de comunicación

Campo	Tamaño (bytes)	Descripción
Cabecera	1	Valor fijo: 0xAA
ID Emisor	1	Identificador del emisor
ID Receptor	1	Identificador del receptor
Línea de imagen	3	Datos binarizados de 3 píxeles
Checksum	1	Suma módulo 256 de los bytes previos



Estructura del paquete (7 bytes)



Figura 1: Arquitectura del sistema de comunicación inalámbrica. Se muestra el flujo interno del emisor, la transmisión RF a 433 MHz, la recepción y la estructura del paquete de 7 bytes.

6. Código ejemplo emisor y receptor

Los códigos base están disponibles en la plataforma Siveduc. A continuación se presenta el esquema de las funciones principales utilizando la librería **RadioHead** (RH_ASK).

Algoritmo 1: Proceso de transmisión – Nodo Emisor

Input: Imagen binarizada I de 32×32 píxeles, ID_TX, ID_RX

Output: Secuencia de paquetes transmitidos por RF

```
1 Inicializar driver RF;
2 for  $fila \leftarrow 0$  to 31 do
3   for  $bloque \leftarrow 0$  to 1 do
4     /* Construir paquete de 7 bytes */
5      $pkt[0] \leftarrow 0xAA$  ; /* Cabecera de sincronización */
6      $pkt[1] \leftarrow ID\_TX$  ; /* Identificador emisor */
7      $pkt[2] \leftarrow ID\_RX$  ; /* Identificador receptor */
8      $pkt[3..5] \leftarrow I[fila][bloque \times 3 .. bloque \times 3 + 2]$  ; /* 3 bytes de imagen */
9     /* Calcular checksum */
10     $chk \leftarrow 0$ ;
11    for  $i \leftarrow 0$  to 5 do
12       $chk \leftarrow chk + pkt[i]$ ;
13     $pkt[6] \leftarrow chk \bmod 256$ ;
14    Transmitir  $pkt$  por RF;
15    Esperar confirmación de envío;
16    Retardo de 50 ms;
17 Retardo de 1000 ms antes del siguiente ciclo;
```

Algoritmo 2: Proceso de recepción y validación – Nodo Receptor

Input: Señal RF entrante

Output: Imagen reconstruida desplegada en consola serial

```
1 Inicializar driver RF;
2 Configurar pin LED como salida;
3 while verdadero do
4   if se recibe paquete de 7 bytes then
5     /* Validar cabecera e ID receptor */
6     if  $\text{pkt}[0] \neq 0xAA$  o  $\text{pkt}[2] \neq MI\_ID$  then
7       Descartar paquete;
8       continuar;
9
10    /* Verificar integridad con checksum */
11     $\text{chk} \leftarrow 0$ ;
12    for  $i \leftarrow 0$  to 5 do
13       $\text{chk} \leftarrow \text{chk} + \text{pkt}[i]$ ;
14    if  $\text{chk} \bmod 256 \neq \text{pkt}[6]$  then
15      Reportar error de integridad por consola;
16      Descartar paquete;
17    else
18      /* Paquete válido: procesar */
19      Parpadear LED indicador;
20
21      /* Desplegar 24 píxeles en consola */
22      for  $\text{byte} \leftarrow \text{pkt}[3]$  to  $\text{pkt}[5]$  do
23        for  $\text{bit} \leftarrow 7$  to 0 do
24          if bit  $b$  de byte es 1 then
25            Imprimir '#';
26          else
27            Imprimir '.';
28        Imprimir salto de línea;
```

7. Actividad para los estudiantes

1. Montar el circuito con los módulos RF y las placas Arduino.
2. Programar el código emisor a un Arduino y el receptor al otro.
3. Observar la consola serial del receptor y validar que el texto es recibido sin errores.
4. Modificar el texto y verificar el cambio.

8. Tareas

1. Implementar el envío y despliegue en consola de una imagen de 32×32 píxeles.
2. Implementar un LED que parpadee al recibir una línea válida.
3. Agregar un segundo ID receptor y programar el filtro para solo aceptar paquetes con su propio ID.
4. Implementar modo *broadcast*: configurar la dirección 0x00 como destino universal, de modo que cualquier nodo receptor acepte el paquete independiente de su ID_RX.
5. Implementar checksum para validar la integridad del paquete recibido.
6. Introducir errores manualmente en el paquete para verificar la detección de errores.
7. Documentar el proceso para la elaboración de un informe final con resultados y problemas encontrados.

9. Metodología básica recomendada

1. Organizar el grupo y definir roles.
2. Estudiar el material entregado.
3. Diseñar los diagramas y algoritmos de solución.
4. Configurar el software necesario y probar los dispositivos en laboratorio.
5. Elaborar un informe que contemple todo el trabajo realizado.

10. Rúbrica de evaluación

La Tabla 2 detalla los criterios de evaluación del laboratorio. Cada ítem es evaluado de forma independiente durante la sesión de demostración, debiendo el grupo ejecutar las funcionalidades.

Cuadro 2: Rúbrica de funcionamiento del sistema de comunicación inalámbrica

Núm.	Criterio	Descripción	Puntaje
1	Transmisión y recepción propia	El grupo demuestra el envío de la imagen 32×32 desde su nodo emisor y la recepción correcta en su propio nodo receptor, desplegando la imagen en consola serial sin errores visibles.	20 pts
2	Recepción desde otro grupo	El nodo receptor del grupo es capaz de recibir y desplegar correctamente una transmisión proveniente de un grupo distinto, configurando su ID_RX para aceptar los paquetes del emisor externo.	20 pts
3	Filtrado por ID receptor	El sistema descarta correctamente los paquetes cuyo ID_RX no corresponde al identificador propio del nodo. Se verifica introduciendo paquetes con ID incorrecto y comprobando que no se despliegan en consola.	20 pts
4	Modo <i>broadcast</i> (ID_RX = 0x00)	Al configurar el emisor con ID_RX = 0x00, todos los nodos receptores activos en el canal deben aceptar y desplegar el paquete, independiente de su identificador propio. Se verifica con al menos dos receptores simultáneos.	20 pts
5	Detección de errores mediante checksum	Se introduce un error manual en uno o más bytes del paquete antes de transmitir. El receptor debe detectar la inconsistencia, reportar el error por consola y descartar el paquete sin desplegarlo.	20 pts
Total			100 pts

Condiciones de evaluación

- Cada criterio se evalúa con puntaje completo si la funcionalidad opera correctamente en el primer intento, y con la mitad del puntaje si requiere corrección menor en el momento de la demostración.
- Un criterio se califica con **0 puntos** si la funcionalidad no opera o no es demostrada dentro del tiempo de la sesión.
- El modo *broadcast* requiere que el receptor implemente la condición:

```
if (pkt[2] == MI_ID || pkt[2] == 0x00)
```

de modo que acepte tanto paquetes dirigidos a él como paquetes de difusión general.

- La demostración del ítem 2 se coordina entre grupos durante la sesión; cada grupo debe acordar previamente los identificadores a utilizar con el grupo externo.